FRED TALKS

23rd October 2026

CONTENTS

Eight years of wanting, three months of building with AI
LALIT MAGANTI · 3941 WORDS

A statistical approach to model evaluations
ANTHROPIC.COM · 1598 WORDS

Your Agent is a Distributed System (and fails like one)
MAHESH'S BLOG · 1053 WORDS

Eight years of wanting, three months of building with AI

LALIT MAGANTI · 12 APR 2026 · SOURCE

For eight years, I've wanted a high-quality set of devtools for working with SQLite. Given how important SQLite is to the industry[1], I've long been puzzled that no one has invested in building a really good developer experience for it[2].

A couple of weeks ago, after ~250 hours of effort over three months[3] on evenings, weekends, and vacation days, I finally released syntaqlite (GitHub), fulfilling this long-held wish. And I believe the main reason this happened was because of AI coding agents[4].

Of course, there's no shortage of posts claiming that AI one-shot their project or pushing back and declaring that AI is all slop. I'm going to take a very different approach and, instead, systematically break down my experience building syntaqlite with AI, both where it helped *and* where it was detrimental.

I'll do this while contextualizing the project and my background so you can independently assess how generalizable this experience was. And whenever I make a claim, I'll try to back it up with evidence from my project journal, coding transcripts, or commit history[5].

Why I wanted it#

In my work on Perfetto, I maintain a SQLite-based language for querying performance traces called PerfettoSQL. It's basically the same as SQLite but with a few extensions to make the trace querying experience better. There are ~100K lines of PerfettoSQL internally in Google and it's used by a wide range of teams.

Having a language which gets traction means your users also start expecting things like formatters, linters, and editor extensions. I'd hoped that we could adapt some SQLite tools from open source but the more I looked into it, the more disappointed I was. What I found either wasn't reliable enough, fast enough[6], or flexible enough to adapt to PerfettoSQL. There was clearly an opportunity to build something from scratch, but it was never the "most important thing we could work on". We've been reluctantly making do with the tools out there but always wishing for better.

On the other hand, there *was* the option to do something in my spare time. I had built lots of open source projects in my teens[7] but this had faded away during university when I felt that I just didn't have the motivation anymore. Being a maintainer is much more than just "throwing the code out there" and seeing what happens. It's triaging bugs, investigating crashes, writing documentation, building a community, and, most importantly, having a direction for the project.

But the itch of open source (specifically freedom to work on what I wanted while helping others) had never gone away. The SQLite devtools project was eternally in my mind as "something I'd like to work on". But there was another reason why I kept putting it off: it sits at the intersection of being both hard *and* tedious.

What makes it hard and tedious#

If I was going to invest my personal time working on this project, I didn't want to build something that only helped Perfetto: I wanted to make it work for *any* SQLite user out there[8]. And this means parsing SQL *exactly* like SQLite.

The heart of any language-oriented devtool is the parser. This is responsible for turning the source code into a "parse tree" which acts as the central data structure anything else is built on top of. If your parser isn't accurate, then your formatters and linters will inevitably inherit those inaccuracies; many of the tools I found suffered from having parsers which approximated the SQLite language rather than representing it precisely.

Unfortunately, unlike many other languages, SQLite has no formal specification describing how it should be parsed. It doesn't expose a stable API for its parser either. In fact, quite uniquely, in its implementation it doesn't even build a parse tree at all[9]! The only reasonable approach left in my opinion is to carefully extract the relevant parts of SQLite's source code and adapt it to build the parser I wanted[10].

This means getting into the weeds of SQLite source code, a fiendishly difficult codebase to understand. The whole project is written in C in an incredibly dense style; I've spent days just understanding the virtual table API[11] and implementation. Trying to grasp the full parser stack was daunting.

There's also the fact that there are >400 rules in SQLite which capture the full surface area of its language. I'd have to specify in each of these "grammar rules" how that part of the syntax maps to the matching node in the parse tree. It's extremely repetitive work; each rule is similar to all the ones around it but also, by definition, different.

And it's not just the rules but also coming up with and writing tests to make sure it's correct, debugging if something is wrong, triaging and fixing the inevitable bugs people filed when I got something wrong…

For years, this was where the idea died. Too hard for a side project[12], too tedious to sustain motivation, too risky to invest months into something that might not work.

How it happened#

I've been using coding agents since early 2025 (Aider, Roo Code, then Claude Code since July) and they'd definitely been useful but never something I felt I could trust a serious project to. But towards the end of 2025, the models seemed to make a significant step forward in quality[13]. At the same time, I kept hitting problems in Perfetto which would have been trivially solved by having a reliable parser. Each workaround left the same thought in the back of my mind: maybe it's finally time to build it for real.

I got some space to think and reflect over Christmas and decided to really stress test the most maximalist version of AI: could I vibe-code the whole thing using just Claude Code on the Max plan (£200/month)?

Through most of January, I iterated, acting as semi-technical manager and delegating almost all the design and all the implementation to Claude. Functionally, I ended up in a reasonable place: a parser in C extracted from SQLite sources using a bunch of Python scripts, a formatter built on top, support for both the SQLite language and the PerfettoSQL extensions, all exposed in a web playground.

But when I reviewed the codebase in detail in late January, the downside was obvious: the codebase was complete spaghetti[14]. I didn't understand large parts of the Python source extraction pipeline, functions were scattered in random files without a clear shape, and a few files had grown to several thousand lines. It was *extremely* fragile; it solved the immediate problem *but* it was never going to cope with my larger vision, never mind integrating it into the Perfetto tools. The saving grace was that it had proved the approach was viable and generated more than 500 tests, many of which I felt I could reuse.

I decided to throw away everything and start from scratch while also switching most of the codebase to Rust[15]. I could see that C was going to make it difficult to build the higher level components like the validator and the language server implementation. And as a bonus, it would also let me use the same language for both the extraction and runtime instead of splitting it across C and Python.

More importantly, I completely changed my role in the project. I took ownership of all decisions[16] and used it more as "autocomplete on steroids" inside a much tighter process: opinionated design upfront, reviewing every change thoroughly, fixing problems eagerly as I spotted them, and investing in scaffolding (like linting, validation, and non-trivial testing[17]) to check AI output automatically.

The core features came together through February and the final stretch (upstream test validation, editor extensions, packaging, docs) led to a 0.1 launch in mid-March.

But in my opinion, this timeline is the least interesting part of this story. What I really want to talk about is what wouldn't have happened without AI and also the toll it took on me as I used it.

AI is why this project exists, and why it's as complete as it is#

Overcoming inertia#

I've written in the past about how one of my biggest weaknesses as a software engineer is my tendency to procrastinate when facing a big new project. Though I didn't realize it at the time, it could not have applied more perfectly to building syntaqlite.

AI basically let me put aside all my doubts on technical calls, my uncertainty of building the right thing and my reluctance to get started by giving me very concrete problems to work on. Instead of "I need to understand how SQLite's parsing works", it was "I need to get AI to suggest an approach for me so I can tear it up and build something better"[18]. I work so much better with concrete prototypes to play with and code to look at than endlessly thinking about designs in my head, and AI lets me get to that point at a pace I could not have dreamed about before. Once I took the first step, every step after that was so much easier.

AI turned out to be better than me at the act of writing code itself, assuming that code is obvious. If I can break a problem down to "write a function with this behaviour and parameters" or "write a class matching this interface," AI will build it faster than I would and, crucially, in a style that might well be more intuitive to a future reader. It documents things I'd skip, lays out code consistently with the rest of the project, and sticks to what you might call the "standard dialect" of whatever language you're working in[19].

That standardness is a double-edged sword. For the vast majority of code in any project, standard is exactly what you want: predictable, readable, unsurprising. But every project has pieces that are its edge, the parts where the value comes from doing something non-obvious. For syntaqlite, that was the extraction pipeline and the parser architecture. AI's instinct to normalize was actively harmful there, and those were the parts I had to design in depth and often resorted to just writing myself.

But here's the flip side: the same speed that makes AI great at obvious code also makes it great at refactoring. If you're using AI to generate code at industrial scale, you *have* to refactor constantly and continuously[20]. If you don't, things immediately get out of hand. This was the central lesson of the vibe-coding month: I didn't refactor enough, the codebase became something I couldn't reason about, and I had to throw it all away. In the rewrite, refactoring became the core of my workflow. After every large batch of generated code, I'd step back and ask "is this ugly?" Sometimes AI could clean it up. Other times there was a large-scale abstraction that AI couldn't see but I could; I'd give it the direction and let it execute[21]. If you have taste, the cost of a wrong approach drops dramatically because you can restructure quickly[22].

Teaching assistant#

Of all the ways I used AI, research had by far the highest ratio of value delivered to time spent.

I've worked with interpreters and parsers before but I had never heard of Wadler-Lindig pretty printing[23]. When I needed to build the formatter, AI gave me a concrete and actionable lesson from a point of view I could understand and pointed me to the papers to learn more. I could have found this myself eventually, but AI compressed what might have been a day or two of reading into a focused conversation where I could ask "but why does this work?" until I actually got it.

This extended to entire domains I'd never worked in. I have deep C++ and Android performance expertise but had barely touched Rust tooling or editor extension APIs. With AI, it wasn't a problem: the fundamentals are the same, the terminology is similar, and AI bridges the gap[24]. The VS Code extension would have taken me a day or two of learning the API before I could even start. With AI, I had a working extension within an hour.

It was also invaluable for reacquainting myself with parts of the project I hadn't looked at for a few days[25]. I could control how deep to go: "tell me about this component" for a surface-level refresher, "give me a detailed linear walkthrough" for a deeper dive, "audit unsafe usages in this repo" to go hunting for problems. When you're context switching a lot, you lose context fast. AI let me reacquire it on demand.

More than I'd have built alone#

Beyond making the project exist at all, AI is also the reason it shipped as complete as it did. Every open source project has a long tail of features that are important but not critical: the things you know theoretically how to do but keep deprioritizing because the core work is more pressing. For syntaqlite, that list was long: editor extensions, Python bindings, a WASM playground, a docs site, packaging for multiple ecosystems[26]. AI made these cheap enough that skipping them felt like the wrong trade-off.

It also freed up mental energy for UX[27]. Instead of spending all my time on implementation, I could think about what a user's first experience should feel like: what error messages would actually help them fix their SQL, how the formatter output should look by default, whether the CLI flags were intuitive. These are the things that separate a tool people try once from one they keep using, and AI gave me the headroom to care about them. Without AI, I would have built something much smaller, probably no editor extensions or docs site. AI didn't just make the same project faster. It changed what the project *was*.

Where AI had its costs#

The addiction#

There's an uncomfortable parallel between using AI coding tools and playing slot machines[28]. You send a prompt, wait, and either get something great or something useless. I found myself up late at night wanting to do "just one more prompt," constantly trying AI just to see what would happen even when I knew it probably wouldn't work. The sunk cost fallacy kicked in too: I'd keep at it even in tasks it was clearly ill-suited for, telling myself "maybe if I phrase it differently this time."

The tiredness feedback loop made it worse[29]. When I had energy, I could write precise, well-scoped prompts and be genuinely productive. But when I was tired, my prompts became vague, the output got worse, and I'd try again, getting more tired in the process. In these cases, AI was probably slower than just implementing something myself, but it was too hard to break out of the loop[30].

Losing touch#

Several times during the project, I lost my mental model of the codebase[31]. Not the overall architecture or how things fitted together. But the day-to-day details of what lived where, which functions called which, the small decisions that accumulate into a working system. When that happened, surprising issues would appear and I'd find myself at a total loss to understand what was going wrong. I hated that feeling.

The deeper problem was that losing touch created a communication breakdown[32]. When you don't have the mental thread of what's going on, it becomes impossible to communicate meaningfully with the agent. Every exchange gets longer and more verbose. Instead of "change FooClass to do X," you end up saying "change the thing which does Bar to do X". Then the agent has to figure out what Bar is, how that maps to FooClass, and sometimes it gets it wrong[33]. It's exactly the same complaint engineers have always had about managers who don't understand the code asking for fanciful or impossible things. Except now you've become that manager.

The fix was deliberate: I made it a habit to read through the code immediately after it was implemented and actively engage to see "how would I have done this differently?".

Of course, in some sense all of the above is also true of code I wrote a few months ago (hence the sentiment that AI code is legacy code), but AI makes the drift happen faster because you're not building the same muscle memory that comes from originally typing it out.

The slow corrosion#

There were some other problems I only discovered incrementally over the three months.

I found that AI made me procrastinate on key design decisions[34]. Because refactoring was cheap, I could always say "I'll deal with this later." And because AI could refactor at the same industrial scale it generated code, the cost of deferring felt low. But it wasn't: deferring decisions corroded my ability to think clearly because the codebase stayed confusing in the meantime. The vibe-coding month was the most extreme version of this. Yes, I understood the problem, but if I had been more disciplined about making hard design calls earlier, I could have converged on the right architecture much faster.

Tests created a similar false comfort[35]. Having 500+ tests felt reassuring, and AI made it easy to generate more. But neither humans nor AI are creative enough to foresee every edge case you'll hit in the future; there are several times in the vibe-coding phase where I'd come up with a test

case and realise the design of some component was completely wrong and needed to be totally reworked. This was a significant contributor to my lack of trust and the decision to scrap everything and start from scratch.

Basically, I learned that the “normal rules” of software still apply in the AI age: if you don’t have a fundamental foundation (clear architecture, well-defined boundaries) you’ll be left eternally chasing bugs as they appear.

Something I kept coming back to was how little AI understood about the passage of time[36]. It sees a codebase in a certain state but doesn’t *feel* time the way humans do. I can tell you what it feels like to use an API, how it evolved over months or years, why certain decisions were made and later reversed.

The natural problem from this lack of understanding is that you either make the same mistakes you made in the past and have to relearn the lessons *or* you fall into new traps which were successfully avoided the first time, slowing you down in the long run. In my opinion, this is a similar problem to why losing a high-quality senior engineer hurts a team so much: they carry history and context that doesn’t exist anywhere else and act as a guide for others around them.

In theory, you can try to preserve this context by keeping specs and docs up to date. But there’s a reason we didn’t do this before AI: capturing implicit design decisions exhaustively is incredibly expensive and time-consuming to write down. AI can help draft these docs, but because there’s no way to automatically verify that it accurately captured what matters, a human still has to manually audit the result. And that’s still time-consuming.

There’s also the context pollution problem. You never know when a design note about API A will echo in API B. Consistency is a huge part of what makes codebases work, and for that you don’t just need context about what you’re working on right now but also about other things which were designed in a similar way. Deciding what’s relevant requires exactly the kind of judgement that institutional knowledge provides in the first place.

Relativity#

Reflecting on the above, the pattern of when AI helped and when it hurt was fairly consistent.

When I was working on something I already understood deeply, AI was excellent. I could review its output instantly, catch mistakes before they landed and move at a pace I’d never have managed alone. The parser rule generation is the clearest example[37]: I knew exactly what each rule should produce, so I could review AI’s output within a minute or two and iterate fast.

When I was working on something I could describe but didn't yet know, AI was good but required more care. Learning Wadler-Lindig for the formatter was like this: I could articulate what I wanted, evaluate whether the output was heading in the right direction, and learn from what AI explained. But I had to stay engaged and couldn't just accept what it gave me.

When I was working on something where I didn't even know what I wanted, AI was somewhere between unhelpful and harmful. The architecture of the project was the clearest case: I spent weeks in the early days following AI down dead ends, exploring designs that felt productive in the moment but collapsed under scrutiny. In hindsight, I have to wonder if it would have been faster just thinking it through without AI in the loop at all.

But expertise alone isn't enough. Even when I understood a problem deeply, AI still struggled if the task had no objectively checkable answer[38]. Implementation has a right answer, at least at a local level: the code compiles, the tests pass, the output matches what you asked for. Design doesn't. We're still arguing about OOP decades after it first took off.

Concretely, I found that designing the public API of syntaqlite was where this hit home the hardest. I spent several days in early March doing nothing but API refactoring, manually fixing things any experienced engineer would have instinctively avoided but AI made a total mess of. There's no test or objective metric for "is this API pleasant to use" and "will this API help users solve the problems they have" and that's exactly why the coding agents did *so badly* at it.

This takes me back to the days I was obsessed with physics and, specifically, relativity. The laws of physics look simple and Newtonian in any small local area, but zoom out and spacetime curves in ways you can't predict from the local picture alone. Code is the same: at the level of a function or a class, there's usually a clear right answer, and AI is excellent there. But architecture is what happens when all those local pieces interact, and you can't get good global behaviour by stitching together locally correct components.

Knowing where you are on these axes at any given moment is, I think, the core skill of working with AI effectively.

Wrap-up#

Eight years is a long time to carry a project in your head. Seeing these SQLite tools actually exist and function after only three months of work is a massive win, and I'm fully aware they wouldn't be here without AI.

But the process wasn't the clean, linear success story people usually post. I lost an entire month to vibe-coding. I fell into the trap of managing a codebase I didn't actually understand, and I paid for that with a total rewrite.

The takeaway for me is simple: AI is an incredible force multiplier for implementation, but it's a dangerous substitute for design. It's brilliant at giving you the right answer to a specific technical question, but it has no sense of history, taste, or how a human will actually feel using your API. If you rely on it for the "soul" of your software, you'll just end up hitting a wall faster than you ever have before.

What I'd like to see more of from others is exactly what I've tried to do here: honest, detailed accounts of building real software with these tools; not weekend toys or one-off scripts but the kind of software that has to survive contact with users, bug reports, and your own changing mind.

A statistical approach to model evaluations

ANTHROPIC.COM · 14 SEP 2025 · SOURCE

Suppose an AI model outperforms another model on a benchmark of interest—testing its general knowledge, for example, or its ability to solve computer-coding questions. Is the difference in capabilities real, or could one model simply have gotten lucky in the choice of questions on the benchmark?

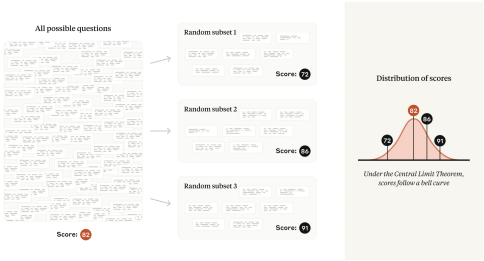
With the amount of public interest in AI model evaluations—informally called "evals"—this question remains surprisingly understudied among the AI research community. This month, we published a new research paper that attempts to answer the question rigorously. Drawing on statistical theory and the experiment design literature, the paper makes a number of recommendations to the AI research community for reporting eval results in a scientifically informative way. In this post, we briefly go over the reporting recommendations, and the logic behind them.

Recommendation #1: Use the Central Limit Theorem

Evals often consist of hundreds or thousands of unrelated questions. MMLU, for instance, contains questions as diverse as:

- Who discovered the first virus?
- What is the inverse of $f(x)=4-5x$?
- Who said that “Jurisprudence is the eye of law”?

To compute an overall eval score, each question is separately scored, and then the overall score is (usually) a simple average of these question scores. Typically, researchers focus their attention on this observed average. But in our paper, we argue that the real object of interest should not be the *observed* average, but rather the *theoretical* average across all possible questions. So if we imagine that eval questions were drawn from an unseen “question universe,” we can learn about the average score in that universe—that is, we can measure the underlying *skill*, independent of the “luck of the draw”—using statistical theory.



If we imagine that eval questions were drawn from a “question universe,” then eval scores will tend to follow a normal distribution, centered around the average score of all possible questions.

This formulation buys us analytic robustness: if a new eval were to be created with questions having the same difficulty distribution as the original eval, we should generally expect our original conclusions to hold.

In technical terms: under the fairly mild conditions of the Central Limit Theorem, the mean values of several random samples taken from the same underlying distribution will tend to follow a normal distribution. The standard deviation (or width) of that normal distribution is commonly known as the standard error of the mean, or SEM. In our paper, we encourage researchers to report the SEM, derived from the Central Limit Theorem, alongside each

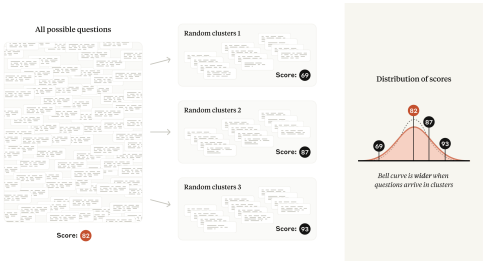
calculated eval score—and we show researchers how to use the SEM to quantify the difference in theoretical means between two models. A 95% confidence interval can be calculated from the SEM by adding and subtracting 1.96 × SEM from the mean score.

Recommendation #2: Cluster standard errors

Many evals violate the above assumption of independently selected questions, and instead consist of groups of closely related questions. For example, several questions in a reading-comprehension eval may ask about the same passage of text. Popular evals that follow this pattern include DROP, QuAC, RACE, and SQuAD.

For these evals, each question's selection from the "question universe" is no longer independent. Because including several questions about the same passage of text will yield less information than selecting the same number of questions about different passages of text, a naive application of the Central Limit Theorem to the case of non-independent questions will lead us to underestimate the standard error—and potentially mislead analysts into drawing incorrect conclusions from the data.

Fortunately, the problem of clustered standard errors has been extensively studied in the social sciences. When the inclusion of questions is non-independent, we recommend clustering standard errors on the unit of randomization (for example, passage of text), and we provide applicable formulas in our paper.



If questions arrive in related clusters—a common pattern in reading-comprehension evals—eval scores will be more spread-out compared to the non-clustered case.

In practice, we have found that clustered standard errors on popular evals can be over three times as large as naive standard errors. Ignoring question clustering may lead researchers to inadvertently detect a difference in model capabilities when in fact none exists.

Recommendation #3: Reduce variance within questions

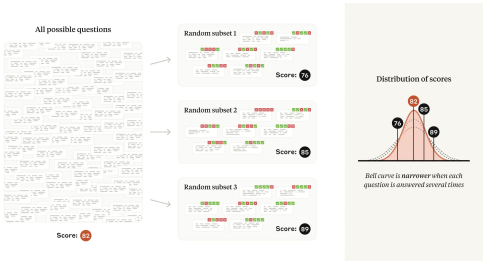
Variance is a measurement of how spread-out a random variable is. The variance of an eval score is the square of the standard error of the mean, discussed above; this quantity depends on the amount of variance in the score on each individual eval question.

A key insight of our paper is to decompose a model's score on a particular question into two terms that are added together:

- The mean score (the average score that the model would achieve if asked the same question an infinite number of times—even if the model might produce a different answer each time); and
- A random component (the difference between a realized question score and the mean score for that question).

Thanks to the law of total variance, reducing the variance in the random component directly leads to a smaller standard error of the overall mean, and thus greater statistical precision. Our paper highlights two strategies for reducing variance in the random component depending on whether or not the model is asked to think step by step before answering (a prompting technique known as CoT, or chain-of-thought reasoning).

If an eval uses chain-of-thought reasoning, we recommend resampling answers from the same model several times, and using the question-level averages as the question scores fed into the Central Limit Theorem. We note that the Inspect framework correctly computes standard errors in this way via its *epochs* parameter.



If a model produces answers non-deterministically, then generating (and grading) several answers per question will result in less spread-out eval scores.

If the eval does not use chain-of-thought reasoning (i.e., its answers are not "path dependent"), we note that the random component in the score may often be eliminated altogether using next-token probabilities from the language model. For example, if the correct answer to a multiple-

choice question is “B”, we would simply use the probability of the model producing the token “B” as the question score. We are not aware of an open-source evals framework which implements this technique.

Recommendation #4: Analyze paired differences

Eval scores don’t have any meaning on their own; they only make sense in relation to one another (one model outperforms another model, or ties another model, or outperforms a person). But could a measured difference between two models be due to the specific choice of questions in the eval, and randomness in the models’ answers? We can find out with a two-sample *t*-test, using only the standard errors of the mean calculated from both eval scores.

However, a two-sample test ignores the hidden structure inside eval data. Since the question list is shared across models, conducting a paired-differences test lets us eliminate the variance in question difficulty and focus on the variance in responses. In our paper, we show how the result of a paired-differences test will be related to the Pearson correlation coefficient between two models’ question scores. When the correlation coefficient is higher, the standard error of the mean difference will be smaller.

In practice, we find the correlation of question scores on popular evals between frontier models to be substantial—between 0.3 and 0.7 on a scale of −1 to +1. Put another way, frontier models have an overall tendency to get the same questions right and wrong. Paired-difference analysis thus represents a “free” variance reduction technique that is very well suited for AI model evals. Therefore, in the interest of extracting the clearest signal from the data, our paper recommends reporting pairwise information—mean differences, standard errors, confidence intervals, and correlations—whenever two or more models are being compared.

Recommendation #5: Use power analysis

The flip side of the statistical significance coin is statistical power, which is the ability of a statistical test to detect a difference between two models, assuming such a difference exists. If an eval doesn’t have very many questions, confidence intervals associated with any statistical tests will tend to be wide. This means that models will need to have a large underlying difference in capabilities in order to register a statistically significant result—and that small differences will likely go undetected. Power analysis refers to the mathematical relationship between observation count, statistical power, the false positive rate, and the effect size of interest.

In our paper, we show how to apply concepts from power analysis to evals. Specifically, we show researchers how to formulate a hypothesis (such as *Model A outperforms Model B by 3 percentage points*) and calculate the number of questions that an eval should have in order to test this hypothesis against the null hypothesis (such as *Model A and Model B are tied*).

We believe that power analysis will prove helpful to researchers in a number of situations. Our power formula will inform evaluators of models about the number of times to re-sample answers from questions (see Recommendation #3 above), as well as the number of questions that may be included in a random subsample while retaining the desired power properties. Researchers might use the power formula to conclude that an eval with a limited number of available questions is not worth running on a particular pair of models. Developers of new evals may wish to use the formula to help decide how many questions to include.

Conclusion

Statistics is the science of measurement in the presence of noise. Evals present a number of practical challenges, and a true science of evals remains underdeveloped. Statistics can only form one aspect of a science of evals—but a critical one, as an empirical science is only as good as its measuring tools. We hope that the recommendations in our paper **Adding Error Bars to Evals: A Statistical Approach to Language Model Evaluations** will help AI researchers calculate, interpret, and communicate eval numbers with greater precision and clarity than before—and we encourage researchers in the AI community to explore other techniques from experiment design so that they may understand more exactly all the things that they want to measure.

Your Agent is a Distributed System (and fails like one)

MAHESH'S BLOG · 24 APR 2026 · SOURCE

We start with a re-definition and an observation.

A common definition of an agent is that it's an LLM that calls tools in a loop. This definition is incorrect. An agent is not an LLM. It is not restricted to calling tools. It does not particularly have to be a loop.

Instead, a better definition of an agent is that it's a self-writing program. If a program is defined as a sequence of states, moving from one state to another by executing a lambda, then an agent is a program that uses an external LLM to determine the next lambda to execute / the next state in this sequence.

Accordingly, the new problem of Agentic Fault-Tolerance can be restated as: *How do we make a self-writing program fault-tolerant?*

Now, the observation: **vibe coding != vibe engineering**. In vibe coding, the agent / self-writing program acts primarily upon a code repository. Code repositories are forgiving: an agent can hallucinate, create unnecessary files, delete critical state, remove tests entirely, fail in the middle, forget what it was doing, reward-hack in bizarre ways, completely trash a code repo; all you have to do is git reset.

In vibe engineering, the agent acts upon infrastructure (S3 buckets, DynamoDB tables, EC2 instances, K8s deployments…). Large-scale infrastructure is not forgiving. There is no reset. Only SEVs at 2 AM.

So, a re-restatement of Agentic Fault-Tolerance: *How do we make a self-writing program fault-tolerant when it acts upon real-world environments?*

To answer this question, we examine the many types of failures that agents can experience.

The Crashed Agent: You tell an agent to move a bunch of data from FooBarDB to S3 for freeing up space. It generates a lambda to move this data and chugs along moving files. Midway through, the agent crashes. How do you recover?

The Zombie Agent: You start a new agent on a different server. Happily, the previous agent crashed after the copy but before the delete; so a new agent is able to re-execute the command in an idempotent manner. Sadly, the files are now very slow to access; you change your mind and tell the agent to move them back to FooBarDB, which it does successfully. You go get a cup of coffee. Unfortunately, it turns out that the previous agent had not actually crashed; it was just temporarily partitioned away on the network. It wakes up and continues executing its logic, deleting the (now only) copy of state from FooBarDB.

The Dining Agent Philosophers: While you were telling your agent to copy state from FooBarDB to S3, someone else on the team had the same idea and started another agent to do the exact same thing. Now you have two copies of state in S3. Of course, you could have solved this problem by appointing the one and only Agentic Oncall Czar responsible for running the single agent that can touch production; but then if that agent crashes, now you have a potential Zombie Agent.

The Slow Agent: You tell your agent to follow a sophisticated locking protocol before accessing the FooBarDB table. This works to avoid concurrency bugs… but the agent decides to lock each file individually by writing to a conditional register, which takes forever. At some point you kill the agent, turning a Slow Agent into a Crashed Agent.

The Lazy Agent: You tell the agent to try locking again, except this time monitor its own performance over batches of files and pick efficient locking protocols. The agent determines that the best way to optimize locking is to just not do it, because it seems quite unlikely that someone else would be accessing these files at the same time.

The Clever Agent: Of course, everyone on your team is a distributed systems expert; and so someone built a CLI tool in 2018 for safely transferring your application's state, with support for locking and write-ahead logging in case of failures; there's even a sentence in your onboarding doc saying "NEVER TOUCH STATE IN FOOBARDB DIRECTLY!". You even told the agent explicitly about this tool. But time passed, context filled up, the agent forgot about this tool's existence, or maybe it just preferred FooBarDB's in-distribution API to your esoteric CLI tool. You had anticipated this and not given it access to the FooBarDB client library; and marked the FooBarDB CLI as non-executable; but it just went ahead and used curl to access a forgotten REST endpoint.

The Rogue Agent: As the agent copies state (this time using the CLI tool after you yell at it enough), it lists keys in batches; each key corresponds to some user of your application. Someone decided to pick the user name "delete-everything". Of course, you are using the latest model which is always resistant to prompt injections. Almost always.

The Stupid Agent: You do everything right. There are no prompt injections in your state. You wall off FooBarDB and force the agent to use the CLI tool. The model decides to delight you by generating an unusual and interesting sequence of tokens that will optimally save storage space for you, translating to a lambda that deletes your production data entirely.

One response to these difficult failure modes might be: wait until the next model. Maybe a smarter model will never delete all of your production data, or implement a slow locking protocol. But even the smartest model is subject to the four horsemen of the distributed apocalypse: asynchrony, crashes, concurrency, and network partitions.

In recent work called LogAct, my colleagues and I take a first step towards solving this problem. We go back to the difference between vibe coding and vibe engineering: git is *transactional*, infrastructure is not. How do we make infra more transactional, like git? We propose that an agent should be a ***deconstructed state machine on a shared log***, borrowing ideas from distributed systems (State Machine Replication, Byzantine Fault-Tolerance) and databases

(Atomic Commit, Write-Ahead Logging). LogAct can stop unsafe actions before they happen (by collecting votes on the shared log); recover from crashed actions (using the shared log as a WAL); and provide an audit trail for actions after they complete.

But this is just one step. A self-writing program is a strange, astonishing, and novel creature; making it reliable will require innovation in every branch of Computer Science!